The Quantum Approximate Optimization Algorithm

From Max-Cut to a CUDA-Q implementation — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

Companion lecture to `QAOA.ipynb`

All quantum results in these slides were computed with NVIDIA CUDA-Q 0.16 (`qpp-cpu`) using the kernel of `QAOA.ipynb`, and checked against an independent NumPy simulation.

What you will learn today

1. What an **optimization problem** is and why combinatorial ones are hard
2. Graphs and the **Max-Cut** problem
3. From bits to spins: the **cost Hamiltonian**
4. The qubit toolkit: Pauli operators, rotations, CNOT
5. The **QAOA circuit**: cost layer, mixer layer, depth p
6. The hybrid loop: **classical optimizer** + quantum expectation values
7. Exact analysis at $p = 1$ and worked examples
8. The CUDA-Q implementation in `QAOA.ipynb`, line by line
9. Results, benchmarks, **limits**, noise and current research

You only need

- school algebra and trigonometry
- vectors and matrices
- basic probability (averages)

Everything quantum is explained on the way.

Three kinds of statements

General theory, **the notebook's convention**, and **illustrative assumptions** are marked as such.

What is an optimization problem?

Optimization

Find the **best** element x^* of a set of candidates $\mathcal{X}$ according to an **objective function** f:

$$x^* = \arg\max_{x \in \mathcal{X}} f(x) \qquad (\text{or } \arg\min).$$

Continuous: $x \in \mathbb{R}^d$ — e.g. the best wing shape, the cheapest mixture of materials. Calculus (gradients) helps.

Combinatorial: x is a discrete choice — a route, a schedule, a yes/no per item. No gradients; the number of candidates explodes.

Examples from engineering: chip placement, network design, scheduling machines, portfolio selection, clustering data.

Why combinatorial optimization is hard

With n yes/no decisions there are 2^n candidates:

n	2^n candidates
10	1 024
15	32 768
30	1.1×10^9
60	1.2×10^{18}
100	1.3×10^{30}

- Trying all candidates (**brute force**) is exact but exponential.
- Many such problems are **NP-hard**: no polynomial-time exact algorithm is known.
- In practice we accept **good** answers quickly: heuristics and **approximation algorithms** with a guaranteed quality.

Approximation ratio

$r = \dfrac{\text{value found}}{\text{optimal value}} \in [0, 1]$ for maximization.

Classical tools for combinatorial problems

Exact

Brute force (2^n), branch and bound, integer programming solvers — exponential in the worst case.

Heuristics (no guarantee)

Greedy local search, simulated annealing, tabu search, genetic algorithms.

Approximation algorithms (guarantee)

For Max-Cut, **Goemans–Williamson** (1995): semidefinite relaxation + random rounding gives $\mathbb{E}[C] \geq 0.878\, C_{\max}$ in polynomial time.

In `QAOA.ipynb`

Exact brute force up to 22 nodes; above that a greedy *local search* with 200 restarts, labelled "NOT guaranteed optimal".

Graph theory in five minutes

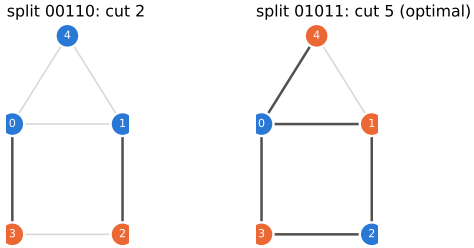
- A **graph** $G = (V, E)$: nodes $V = \{0, \ldots, n-1\}$ and edges $E \subseteq \{(i, j)\}$.
- A **weighted** graph gives every edge a number w_{ij}; $W = \sum_{(i,j)\in E} w_{ij}$.
- **Degree** d_i = number of edges at node i. **Handshake lemma:** $\sum_i d_i = 2|E|$.
- A **cycle** returns to its start; it is **odd** if it has an odd number of edges.
- **Bipartite**: nodes can be 2-coloured so that every edge joins different colours $\iff$ no odd cycle.

Standard families

- ring (cycle) C_n: n edges, $d = 2$
- complete graph K_n: $\binom{n}{2}$ edges
- random graph $G(n, q)$: each pair is an edge with probability q; $\mathbb{E}|E| = \binom{n}{2} q$

The notebook's default: $G(15, 0.3)$ with `SEED = 42` $\Rightarrow$ 30 edges (expected 31.5).

The Max-Cut problem



"House graph": square 0−1−2−3 plus roof node 4.

Max-Cut

Split the nodes into two groups. The **cut** is the total weight of edges *between* the groups. Find the split with the largest cut.

A split is a bitstring $z = z_0 z_1 \dots z_{n-1}$, $z_i \in \{0, 1\}$ = group of node i:

$$C(z) = \sum_{(i,j)\in E} w_{ij} \, [z_i \neq z_j].$$

$[\cdot]$ is 1 if the condition holds, else 0. Dark edges in the figure are cut.

Worked example: the house graph

1. **Upper bound.** Nodes $0, 1, 4$ form a triangle (odd cycle): at least one of its edges stays uncut, so $C \leq 6 - 1 = 5$.
2. **Construction.** Colour the square alternately: $z_0 z_1 z_2 z_3 = 0101$ cuts all 4 square edges. Node 4 sees node 0 (group 0) and node 1 (group 1): exactly one roof edge is cut, whatever z_4 is. Total $4 + 1 = 5$.
3. **Counting.** $z_0 z_1 z_2 z_3 \in \{0101, 1010\}$ and z_4 free: **4 optimal bitstrings**, but only **2 different partitions** (a split and its complement $\bar{z}$ are the same partition).

Random guessing

Each edge is cut with probability $\frac{1}{2}$ by a random split, so $\mathbb{E}[C] = W/2 = 3$ and $r_{\text{random}} = 3/5 = 0.6$. $P(\text{random split is optimal}) = 4/32 = 1/8$.

Facts about cuts you will use all the time

- $C(z) = C(\bar{z})$: every partition appears as **two** bitstrings; 2^n strings, 2^{n-1} partitions.
- $C_{\max} = |E| \iff$ the graph is bipartite. Odd cycles force uncut edges:
  $C_{\max}(C_n) = n - 1$ (n odd),
  $C_{\max}(K_n) = \lfloor \frac{n}{2} \rfloor \lceil \frac{n}{2} \rceil$.
- A uniformly random split:
  $\mathbb{E}[C] = W/2$ and (unit weights)
  $\mathrm{Var}(C) = |E|/4$, because the edge indicators are pairwise independent.
- Hence $C_{\max} \geq |E|/2$ for every graph.

The notebook's 15-node graph

$|E| = 30$, $W/2 = 15$, $\mathrm{Var}(C) = 7.5$ ($\sigma = 2.74$).
Brute force: $C_{\max} = 25$, reached by 4 bitstrings (2 partitions).

$C_{\max}$ lies $(25 - 15)/2.74 \approx 3.7\,\sigma$ above the random mean.

Complexity

Max-Cut is NP-hard; *minimum* cut is polynomial (max-flow / Stoer–Wagner).

Where Max-Cut appears

- **VLSI design**: assign wires to two layers to minimise vias.
- **Statistical physics**: ground state of an Ising spin glass = a weighted Max-Cut.
- **Clustering / partitioning**: split data or networks into two strongly opposed groups.
- **QUBO**: every quadratic unconstrained binary optimisation problem has the same mathematical form.

Why Max-Cut for quantum optimization?

- simple to state and to check
- its cost function has only two-body terms $\Rightarrow$ two-qubit gates
- it is the benchmark used in the original QAOA paper (Farhi, Goldstone, Gutmann 2014)

Step 1: bits become spins

Replace each bit by a **spin** $s_i = (-1)^{z_i} = 1 - 2z_i$: $\quad z_i = 0 \mapsto s_i = +1$, $z_i = 1 \mapsto s_i = -1$.

The key identity

$$[z_i \neq z_j] = \frac{1 - s_i s_j}{2} \qquad \text{(check: equal spins } \Rightarrow s_i s_j = +1 \Rightarrow 0;\ \text{different} \Rightarrow -1 \Rightarrow 1)$$

Summing over the edges:

$$C(z) = \sum_{(i,j)} w_{ij} \frac{1 - s_i s_j}{2} = \underbrace{\frac{W}{2}}_{\text{constant}} \underbrace{- \sum_{(i,j)} \frac{w_{ij}}{2} s_i s_j}_{\text{Ising energy}}.$$

Maximising the cut = minimising the Ising energy. In bits the same function is a QUBO: $[z_i \neq z_j] = z_i + z_j - 2z_i z_j$.

Step 2: spins become Pauli-Z operators

On a qubit, $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ has $Z|0\rangle = +|0\rangle$, $Z|1\rangle = -|1\rangle$:
its eigenvalue on $|z_i\rangle$ is exactly the spin s_i.

So $Z_i Z_j |z\rangle = s_i s_j |z\rangle$, and we define

Cost Hamiltonian (notebook convention)

$$H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2} Z_i Z_j, \qquad C = \frac{W}{2} I - H_C.$$

- H_C is **diagonal**: every bitstring $|z\rangle$ is an eigenvector with eigenvalue $H_C(z) = \frac{W}{2} - C(z)$.
- The **ground state** (lowest energy) of H_C is an optimal cut: $E_0 = \frac{W}{2} - C_{\max}$.
- The constant $\frac{W}{2}$ is dropped in the code; it only shifts energies.
- **Sign matters:** the notebook evolves with H_C, many textbooks with $C = \frac{W}{2} - H_C$.

Example: the triangle is frustrated

$H_C = \frac{1}{2}(Z_0Z_1 + Z_1Z_2 + Z_0Z_2)$ on the 8 basis states:

z	$s_0s_1,\ s_1s_2,\ s_0s_2$	$H_C(z)$	$C(z)$
$000, 111$	$+1, +1, +1$	$+\frac{3}{2}$	0
6 others	two -1, one $+1$	$-\frac{1}{2}$	2

- $-\frac{3}{2}$ (all three edges cut) is impossible: $(s_0s_1)(s_1s_2)(s_0s_2) = +1$.
- Ground energy $-\frac{1}{2}$, six-fold degenerate; $C_{\max} = \frac{3}{2} - (-\frac{1}{2}) = 2$.
- This "frustration" of odd cycles is what makes Max-Cut hard in general.

Qubits, superposition and measurement

- A qubit: $|\psi\rangle = a\,|0\rangle + b\,|1\rangle$, $|a|^2 + |b|^2 = 1$.
- n qubits: $|\psi\rangle = \sum_{z\in\{0,1\}^n} a_z\,|z\rangle$ — 2^n complex amplitudes.
- **Born rule**: measuring gives the bitstring z with probability $|a_z|^2$.
- **Expectation value** of a diagonal observable: $\langle H_C\rangle = \sum_z |a_z|^2 H_C(z)$.

Uniform superposition

$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_z |z\rangle$: every split equally likely — **random guessing**, $\langle C\rangle = W/2$.

15 qubits

$2^{15} = 32\,768$ amplitudes; a state vector in fp64 needs 16×2^{15} bytes $= 524$ KB.

In $|+\rangle^{\otimes 15}$ the 4 optimal bitstrings have total probability $4/32\,768 = 1.22 \times 10^{-4}$.

Relative phases

Phases $e^{i\phi}$ on amplitudes do not change $|a_z|^2$; they matter only after *interference*.

Pauli operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad X^2 = Y^2 = Z^2 = I.$$

Multiplication table

$XY = iZ$, $YZ = iX$, $ZX = iY$;
reversing the order flips the sign:
$YX = -iZ$, ...

- Different Paulis **anticommute**: $XZ = -ZX$.
- Pauli strings (Z_0Z_1, X_0X_1, ...) commute iff they anticommute at an *even* number of positions.

Roles in QAOA

- Z_iZ_j: measures whether edge (i, j) is cut (cost)
- X_i: flips bit i (mixer: moves probability between splits)
- Y: appears in the analysis (Heisenberg picture)

All Z_iZ_j commute with each other; X_i anticommutes with Z_iZ_j.

Rotation gates and the exponential of a Pauli

One formula for all rotations

If $P^2 = I$ then $\quad e^{-i\phi P} = \cos\phi\, I - i \sin\phi\, P \quad$ (expand the power series).

Half-angle convention (CUDA-Q, textbooks):

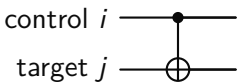
$$R_X(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$$

$$R_Z(\theta) = e^{-i\theta Z/2} = \mathrm{diag}\left(e^{-i\theta/2}, e^{i\theta/2}\right)$$

- $R_X(2\beta) = e^{-i\beta X}$ — hence `rx(2.0 * beta)` in the notebook.
- $R_X(2\beta)\,|0\rangle = \cos\beta\,|0\rangle - i\sin\beta\,|1\rangle$: flips with probability $\sin^2\beta$.
- $e^{-i\frac{\pi}{2}X} = -iX$: a full bit flip (up to phase).
- R_Z only adds phases: invisible in a Z-measurement.

CNOT and how it moves Pauli operators

CNOT flips the target if the control is 1: $|a, b\rangle \mapsto |a, a \oplus b\rangle$.



Conjugation rules

$X \otimes I \to X \otimes X$ (X: control $\to$ target)
$I \otimes Z \to Z \otimes Z$ (Z: target $\to$ control)
$Z \otimes I$ and $I \otimes X$ unchanged

Consequence: a ZZ rotation from one R_Z

$$\mathrm{CNOT}\,(I \otimes R_Z(\theta))\,\mathrm{CNOT} = e^{-i\theta\, Z\otimes Z/2}$$

Proof on a basis state: CNOT writes $z_i \oplus z_j$ on qubit j; $R_Z(\theta)$ gives $e^{\mp i\theta/2}$ for parity 0/1; the second CNOT restores z_j.

- $e^{-i\theta Z\otimes Z/2} = \mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2}, e^{i\theta/2}, e^{-i\theta/2})$
- equal bits get $e^{-i\theta/2}$, different bits $e^{+i\theta/2}$

Expectation values and the Heisenberg picture

For a state $|\psi\rangle = U|\psi_0\rangle$:

$$\langle O\rangle = \langle\psi_0| U^\dagger O U |\psi_0\rangle .$$

Instead of evolving the state we can evolve the **operator** $O \to U^\dagger O U$ (Heisenberg picture).

The rule used for the mixer

$$e^{i\beta X} Z e^{-i\beta X} = \cos 2\beta\, Z + \sin 2\beta\, Y$$

(move Z through: $e^{i\beta X} Z = Z e^{-i\beta X}$; then $ZX = iY$).

- Linearity: $\langle H_C\rangle = \sum_{(i,j)} \frac{w_{ij}}{2}\langle Z_i Z_j\rangle$ — `cudaq.observe` evaluates this term by term.
- Per edge: $\langle C_{ij}\rangle = \frac{1}{2}(1 - \langle Z_i Z_j\rangle) = P(\text{edge cut})$.
- From shots: average $(-1)^{z_i \oplus z_j}$; standard error $\leq 1/\sqrt{S}$.

The idea of QAOA

1. Start in $|+\rangle^{\otimes n}$: all splits equally likely.
2. **Cost layer** $e^{-i\gamma H_C}$: give every split a phase that depends on its cut value.
3. **Mixer layer** $e^{-i\beta H_M}$, $H_M = \sum_i X_i$: let amplitudes of neighbouring splits (one bit apart) **interfere**, so that good splits gain probability.
4. Repeat p times with angles γ_l, β_l.
5. A **classical optimizer** tunes the $2p$ angles to maximise $\langle C \rangle$.
6. **Sample** the final circuit: each shot is a candidate split.

Motivation: quantum annealing

A slow change from a mixer Hamiltonian to the cost Hamiltonian keeps the system in an extremal eigenstate (adiabatic theorem). Cutting this evolution into small alternating steps (Trotterisation) gives exactly the QAOA structure; QAOA keeps the structure but **optimises the step sizes**.

The QAOA state

Depth-p QAOA (Farhi, Goldstone, Gutmann 2014) — notebook convention

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C}}_{\text{layer } p} \cdots \underbrace{e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C}}_{\text{layer 1}} |+\rangle^{\otimes n}$$

with $H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2} Z_i Z_j$ and $H_M = \sum_i X_i$.

- $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$: cost angles; $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$: mixer angles.
- Operator order: layer 1 acts *first* (rightmost).
- $2p$ real parameters, regardless of n.

Objective

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | C | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle$$
$$= \frac{W}{2} - \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | H_C | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle$$

Approximation ratio $r = F_p / C_{\max}$.

Why the initial state is $|+\rangle^{\otimes n}$

- $H|0\rangle = |+\rangle$, so one layer of Hadamards prepares it from $|0\rangle^{\otimes n}$ (code: `h(qubits)`).
- It is unbiased: no split is preferred.
- $X|+\rangle = |+\rangle$, so $H_M|+\rangle^{\otimes n} = n|+\rangle^{\otimes n}$: it is the eigenstate of H_M with the **largest** eigenvalue $+n$ (ground state of $-H_M$).
- Therefore a mixer acting alone does nothing: if all $\gamma_l = 0$ then $F_p = W/2$.

Annealing direction

Adiabatic evolution keeps a state at the same *position* of the spectrum. Starting from the *top* state of $+H_M$, the matching target is the top state of the final Hamiltonian. To end in the *ground* state of H_C (max cut), interpolate $H_M \to -H_C$: in the notebook's convention γ and β should have **opposite signs**.

The cost unitary $U_C(\gamma) = e^{-i\gamma H_C}$

- All Z_iZ_j commute $\Rightarrow$ exact factorisation, any order:

$$e^{-i\gamma H_C} = \prod_{(i,j)\in E} e^{-i\gamma \frac{w_{ij}}{2} Z_iZ_j}$$

- Each factor is diagonal: $|z\rangle \mapsto e^{-i\gamma \frac{w_{ij}}{2} s_i s_j} |z\rangle$.
- Overall: $|z\rangle \mapsto e^{-i\gamma H_C(z)} |z\rangle = e^{-i\gamma W/2}\, e^{+i\gamma C(z)} |z\rangle$.

The cost layer writes the cut value into the phase. It does not change any probability by itself.

Gate decomposition per edge

$$e^{-i\gamma \frac{w}{2} Z_iZ_j} = \mathrm{CNOT}_{ij}\; R_Z^{(j)}(\gamma w)\; \mathrm{CNOT}_{ij}$$

2 CNOTs + 1 R_Z. The angle is γw (not $\gamma w/2$) because $R_Z(\theta) = e^{-i\theta Z/2}$.

Cost layer: $2|E|$ CNOTs and $|E|$ R_Z gates. The cost layer is the only entangling part of QAOA.

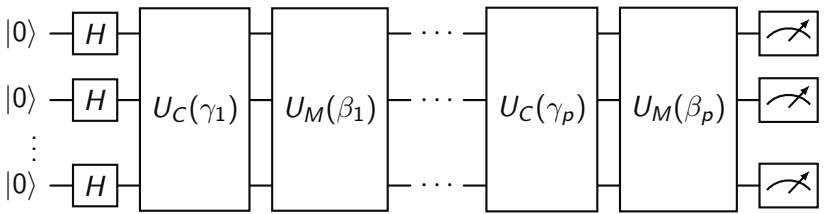
The mixer unitary $U_M(\beta) = e^{-i\beta H_M}$

- The X_i commute: $e^{-i\beta H_M} = \prod_i e^{-i\beta X_i} = \bigotimes_i R_X(2\beta)$.
- Single-qubit gates only: n gates, no entanglement.
- $e^{-i\beta X}$ mixes $|0\rangle$ and $|1\rangle$: amplitude flows between splits that differ in one bit — this is where the phases from the cost layer **interfere**.
- H_M does not commute with H_C: without that, nothing could change.

Symmetries of F_p

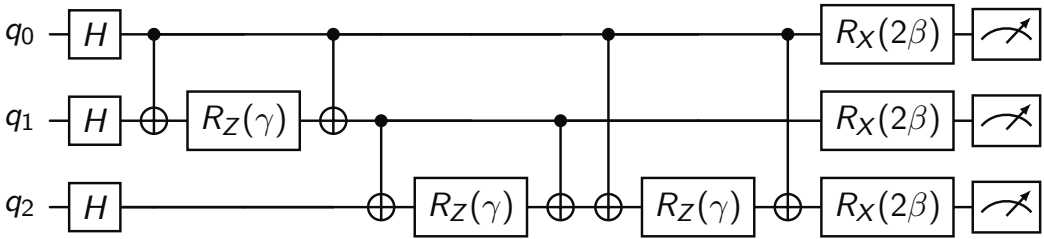
- $\beta_l \to \beta_l + \frac{\pi}{2}$: inserts $(-i)^n X^{\otimes n}$ (flip all bits); $C(\bar{z}) = C(z) \Rightarrow$ **period** $\pi/2$ in each β_l.
- $(\boldsymbol{\gamma}, \boldsymbol{\beta}) \to (-\boldsymbol{\gamma}, -\boldsymbol{\beta})$: complex conjugate state, same probabilities $\Rightarrow F_p$ unchanged.
- integer weights: F_p is 2π-periodic in each γ_l.

The QAOA circuit



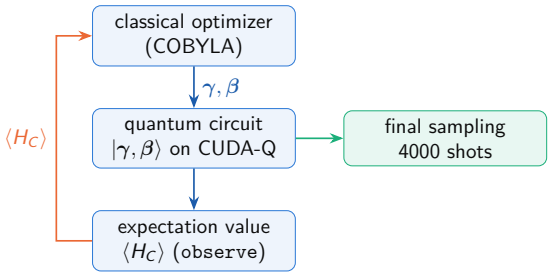
- $U_C(\gamma)$: for every edge CNOT–$R_Z(\gamma w)$–CNOT. $U_M(\beta)$: $R_X(2\beta)$ on every qubit.
- Gate count: $n + p\,(3|E| + n)$; two-qubit gates $2p|E|$. Default ($n = 15$, $|E| = 30$, $p = 2$): **225 gates, 120 CNOTs**, depth 64 (recorded statistics).
- The final measurement in the Z basis returns a bitstring = a candidate split.

One layer on the triangle, gate by gate



- Edges in list order $(0,1),(1,2),(0,2)$, unit weights; CNOT control = first node, target = second node, R_Z on the target — exactly as `x.ctrl(q[i], q[j]); rz(gamma * w, q[j]); x.ctrl(q[i], q[j])`.
- $3 + 3 \cdot 3 + 3 = 15$ gates, 6 CNOTs. The notebook draws such a one-layer circuit for the first 10 nodes.

Parameters, depth and the hybrid loop



- Angle vector (notebook): $\texttt{params} = [\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p]$ — blocked, not interleaved.
- Depth p: more layers can only help with *optimal* angles ($F^*_{p+1} \geq F^*_p$, set the extra angles to 0), but more parameters are harder to optimise.
- $p \to \infty$: $F^*_p \to C_{\max}$ (adiabatic limit).
- Each optimizer step = one full quantum simulation.

QAOA in pseudocode

```
input : graph G = (V, E, w), depth p, initial angles theta0 = (gamma_1..p, beta_1..p)
H_C <- sum over edges (w_ij / 2) Z_i Z_j # cost Hamiltonian
define prepare(theta): # the parameterised circuit
    apply H to every qubit # |+>^n
    for l = 1..p:
        for (i, j, w) in E: CNOT(i, j); RZ(gamma_l * w) on j; CNOT(i, j)
        for q in V: RX(2 * beta_l) on q
define objective(theta): return <prepare(theta)| H_C |prepare(theta)>
theta* <- classical_minimise(objective, theta0) # COBYLA, 80 evaluations
samples <- measure prepare(theta*) S times # S = 4000
return the sampled bitstring z with the largest cut C(z), and <C> = W/2 - objective(theta*)
```

The best *evaluated* angles are used for sampling (the notebook keeps the history, not only the final point).

$p = 1$ on a single edge: derivation (1/2)

Goal: $\langle Z_0 Z_1 \rangle = \langle {+}{+}| \, U_C^\dagger U_M^\dagger \, Z_0 Z_1 \, U_M U_C \, |{+}{+}\rangle$, $U_C = e^{-i\gamma Z_0 Z_1/2}$, $U_M = e^{-i\beta(X_0+X_1)}$.

Step 1: through the mixer (each qubit separately)

With $c = \cos 2\beta$, $s = \sin 2\beta$:

$$U_M^\dagger Z_0 Z_1 U_M = (cZ_0 + sY_0)(cZ_1 + sY_1) = c^2 Z_0 Z_1 + cs\,(Z_0 Y_1 + Y_0 Z_1) + s^2 Y_0 Y_1.$$

Step 2: through the cost gate

$Z_0 Z_1$ and $Y_0 Y_1$ commute with $Z_0 Z_1$: unchanged. $Y_0 Z_1$ anticommutes, so

$$U_C^\dagger (Y_0 Z_1) U_C = Y_0 Z_1 e^{-i\gamma Z_0 Z_1} = \cos\gamma \; Y_0 Z_1 - i \sin\gamma \; Y_0 Z_0 = \cos\gamma \; Y_0 Z_1 + \sin\gamma \; X_0$$

(using $YZ = iX$); likewise $Z_0 Y_1 \to \cos\gamma \, Z_0 Y_1 + \sin\gamma \, X_1$.

$p = 1$ on a single edge: derivation (2/2)

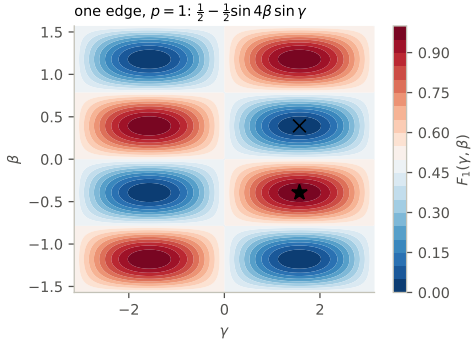
Step 3: evaluate in $|{+}{+}\rangle$

$\langle X_0 \rangle = \langle X_1 \rangle = 1$; every term with a Y or a single Z has expectation 0;
$\langle Z_0 Z_1 \rangle = \langle Y_0 Y_1 \rangle = 0$.

$$\langle Z_0 Z_1 \rangle = 2cs \sin\gamma = \sin 4\beta \, \sin\gamma$$

$$F_1(\gamma, \beta) = \tfrac{1}{2}(1 - \sin 4\beta \, \sin\gamma)$$

- Maximum $F_1 = 1$ at $\sin 4\beta \sin\gamma = -1$, e.g. $(\gamma, \beta) = (\frac{\pi}{2}, -\frac{\pi}{8})$.
- At $(\frac{\pi}{2}, +\frac{\pi}{8})$: $F_1 = 0$ — the edge is **never** cut.



$\star$ optimum, $\times$ worst point. CUDA-Q values agree with the formula to 10^{-9}.

Sign conventions: textbook versus notebook

Two common conventions

textbook (Farhi et al.): $e^{-i\gamma C}$
notebook: $e^{-i\gamma H_C}$ with $H_C = \frac{W}{2} - C$
$e^{-i\gamma C} = e^{-i\gamma W/2} e^{+i\gamma H_C}$: the same circuit with $\gamma \to -\gamma$.

Single edge, textbook form:
$F_1 = \frac{1}{2} + \frac{1}{2} \sin 4\beta \sin \gamma$ — optimum at $(\frac{\pi}{2}, \frac{\pi}{8})$, the *worst* point in the notebook's convention.

Rule of thumb

Always translate literature angles before using them in `QAOA.ipynb`: flip the sign of every γ (equivalently of every β). Adding a constant to H_C is harmless (global phase); a sign change is not.

Small-angle check

One edge: $\Delta F_1 \approx -2\beta\gamma$. Positive γ and positive β *lower* the cut.

$p = 1$ for triangle-free graphs

The same Heisenberg calculation for an edge (u, v) whose end nodes have degrees d_u, d_v and no common neighbour gives (notebook convention; Wang, Hadfield, Jiang, Rieffel 2018 in the textbook convention):

Edge correlator

$$\langle Z_u Z_v \rangle = \tfrac{1}{2} \sin 4\beta \, \sin\gamma \, (\cos^{d_u - 1}\gamma + \cos^{d_v - 1}\gamma), \qquad \langle C_{uv} \rangle = \tfrac{1}{2}(1 - \langle Z_u Z_v \rangle)$$

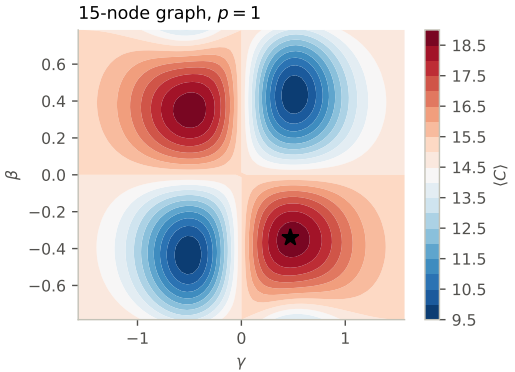
Ring C_n ($d = 2$)

$\langle C_{uv} \rangle = \frac{1}{2} - \frac{1}{4} \sin 4\beta \sin 2\gamma \leq \frac{3}{4}$ at $(\frac{\pi}{4}, -\frac{\pi}{8})$.
Even ring: $r = 3/4$ (numerical optimum: 0.7500).

Single edge ($d = 1$)

$\cos^0 \gamma = 1$: recovers $\sin 4\beta \sin\gamma$.
3-regular graphs: $r \geq 0.6924$ at $p = 1$ (Farhi et al. 2014).

The $p = 1$ landscape of the notebook's graph



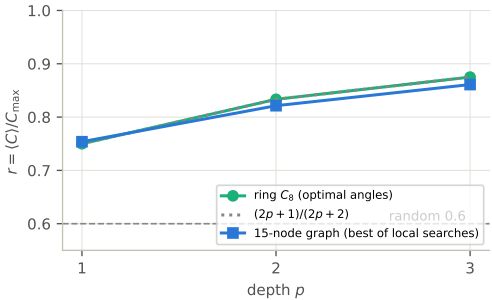
- $\langle C \rangle$ on a 61×61 grid of (γ, β) (NumPy; the best point re-checked with CUDA-Q).
- Best grid point: $\langle C \rangle = 18.84$ at $(\gamma, \beta) \approx (0.47, -0.34)$ — again γ and β of **opposite signs**.
- The point symmetry $(\gamma, \beta) \to (-\gamma, -\beta)$ is visible; blue regions are *worse* than random.
- $p = 2$ adds two more angles and reaches 20.53 (Part 9).

Depth, light cones and the ring

Light cone

In $U^\dagger Z_u Z_v U$ every cost layer adds the neighbours of the current support; gates further away cancel. After p layers $\langle Z_u Z_v \rangle$ depends only on the nodes within distance p of the edge — on a ring, $2p+2$ qubits.

- Fixed-p QAOA is a **local** algorithm.
- Rings with $n \geq 2p+2$: every edge sees the same path, so r does not depend on n.
- Ring, optimised numerically: $p = 1, 2, 3$: 0.7500, 0.8333, 0.8750 $= \frac{3}{4}, \frac{5}{6}, \frac{7}{8}$.



15-node graph: best values found by local optimisation from several starts (not proven global): $p = 1, 2, 3$: 0.754, 0.821, 0.861.

Exactly solvable at $p = 1$: the triangle

- H_C on K_3 has only two eigenvalues: $+\frac{3}{2}$ on $\{000, 111\}$, $-\frac{1}{2}$ on the 6 optimal strings.
- So $e^{-i\gamma H_C}$ only sets one relative phase $e^{-2i\gamma}$ between "bad" and "good" strings.
- One mixer can then remove the bad amplitudes completely.

Result (checked with CUDA-Q)

At $\gamma^* = \arctan\frac{1}{\sqrt{2}} \approx 0.6155$, $\beta^* = -\gamma^*/2$: $F_1 = 2 = C_{\max}$, i.e. $r = 1$ and $P_{\text{opt}} = 1$.

Do not generalise

Exact solution at $p = 1$ is special to tiny, highly symmetric graphs (one edge, the triangle). The ring already stops at $\frac{3}{4}$, and general graphs need larger p — with no fixed guarantee.

Worked example 1: one edge, traced amplitude by amplitude

$\gamma = \frac{\pi}{2}$, $\beta = \frac{\pi}{8}$, order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ (notebook convention).

after	$\|00\rangle$	$\|01\rangle$	$\|10\rangle$	$\|11\rangle$
$H \otimes H$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
CNOT, $R_Z(\frac{\pi}{2})$ on q_1, CNOT	$\frac{1}{2}e^{-i\pi/4}$	$\frac{1}{2}e^{+i\pi/4}$	$\frac{1}{2}e^{+i\pi/4}$	$\frac{1}{2}e^{-i\pi/4}$
probabilities so far	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
$R_X(\frac{\pi}{4}) \otimes R_X(\frac{\pi}{4})$: probabilities	0.5	0	0	0.5

- The cost layer gives cut strings the relative phase $\gamma = 90°$; probabilities are unchanged.
- The mixer turns the phases into probabilities: here **all** weight goes to the *uncut* strings, because $\sin 4\beta \sin \gamma = +1$ (formula: $P(\text{cut}) = \frac{1}{2}(1 - \sin 4\beta \sin \gamma) = 0$).
- With $\beta = -\frac{\pi}{8}$ the same circuit returns 01 or 10 with certainty.

Worked example 2: the path $0-1-2$ at $p=1$

$(\gamma,\beta)=(\frac{\pi}{2},-\frac{\pi}{8})$: $\sin\gamma=1$, $\cos\gamma=0$, $\sin 4\beta=-1$.

1. Edge $(0,1)$: $d_0=1$, $d_1=2$: $\langle Z_0Z_1\rangle=\frac{1}{2}(-1)(1)(\cos^0\gamma+\cos^1\gamma)=\frac{1}{2}(-1)(1+0)=-\frac{1}{2}$.
2. Edge $(1,2)$: the same by symmetry.
3. $\langle C\rangle=2\cdot\frac{1}{2}(1+\frac{1}{2})=1.5$, $r=1.5/2=0.75$.

State-vector check

$P(010)=P(101)=0.3125$; each other string 0.0625.
$P_{\text{opt}}=0.625$.
$\langle C\rangle=2(0.625)+1(4\times0.0625)=1.5$ ✓

A sampler finds the optimum with probability 0.625 per shot although the average ratio is only 0.75.

Worked example 3: counting the default circuit

$n = 15$, $|E| = 30$, $p = 2$:

gate	count
H (initial state)	$n = 15$
CNOT (cost)	$2p\|E\| = 120$
R_Z (cost)	$p\|E\| = 60$
R_X (mixer)	$pn = 30$
total (no measurements)	225
parameters	$2p = 4$

These agree with the statistics recorded by the notebook: `h 15, cx 120, rz 60, rx 30, mz 15`, depth 64.

Scaling

Two-qubit gates grow like $2p|E|$; for $G(n, 0.3)$, $|E| \approx 0.15\, n^2$. At $n = 25$, $|E| = 90$: 615 gates, 360 CNOTs per circuit ($p = 2$).

The structure of `QAOA.ipynb`

1. Problem statement and mathematics
2. Imports and version check
3. **Input parameters** (the only cell to edit)
4. Example inputs: build the graph, brute-force reference
5. Benchmark utilities, backend selection
6. Expected output: memory, time estimate
7. Circuit generation, drawing, statistics, QIR export
8. **Execution**: optimisation + sampling, live memory monitor
9. Results and plots
10. Final benchmark report (JSON + PNG in `exports/qaoa/`)

Section 3 defaults

`N_QUBITS = 15`, `GRAPH = "example"`, `EDGE_PROBABILITY = 0.3`
`QAOA_LAYERS = 2`, `OPTIMIZER = "COBYLA"`
`MAX_EVALUATIONS = 80`, `INITIAL_PARAMETERS = "ramp"`
`OBJECTIVE_SHOTS = None` (exact), `SHOTS = 4000`, `SEED = 42`
`BRUTE_FORCE_MAX_NODES = 22`, `BACKEND = "auto"`

Graphs: `"example"` (random), `"ring"`, `"complete"`, `"custom"` (weighted edge list).

Building the instance and the classical reference (Section 4)

```
graph_rng = np.random.default_rng(SEED) # reproducible random graph
edges = [(i, j, 1.0) for i in range(N_QUBITS) for j in range(i + 1, N_QUBITS)
        if graph_rng.random() < EDGE_PROBABILITY] # each pair with probability 0.3
edge_i = [e[0] for e in edges]; edge_j = [e[1] for e in edges]; edge_w = [e[2] for e in edges]
total_weight = float(sum(edge_w)) # W: a random split has <C> = W/2

def brute_force_maxcut(): # all 2^N splits, vectorised
    index = np.arange(2 ** N_QUBITS, dtype=np.int64)
    values = np.zeros(2 ** N_QUBITS)
    for i, j, w in edges: # bit i of z XOR bit j of z
        values += w * (((index >> (N_QUBITS - 1 - i)) ^ (index >> (N_QUBITS - 1 - j))) & 1)
    best = values.max()
    optimal = [format(int(z), f"0{N_QUBITS}b") for z in np.flatnonzero(np.isclose(values, best))]
    return best, optimal
```

Node 0 is the most significant bit and is written first. Output of the default: 30 edges, $W = 30$, max cut 25, 4 optimal splits, 0.01 s. For $N > 22$ a greedy local search replaces brute force.

The cost Hamiltonian and the initial angles (Section 7a)

```
def cost_hamiltonian():
    """H_C = sum_(i,j) w_ij/2 Z_i Z_j (maximising the cut <=> minimising <H_C>)."""
    hamiltonian = None
    for i, j, w in edges:
        term = 0.5 * w * cudaq.spin.z(i) * cudaq.spin.z(j) # one Z_i Z_j term per edge
        hamiltonian = term if hamiltonian is None else hamiltonian + term
    return hamiltonian

if INITIAL_PARAMETERS == "ramp":
    initial_params = np.concatenate([np.linspace(0.2, 0.6, QAOA_LAYERS), # gamma_1..gamma_p rising
                                     np.linspace(0.6, 0.2, QAOA_LAYERS)]) # beta_1..beta_p falling
```

- `cudaq.spin.z(i)` is the operator Z_i; products and sums build a `SpinOperator` with 30 terms.
- No constant: $\langle C \rangle$ is recovered as `total_weight / 2 - energy`.
- $p = 2$ ramp: $\gamma = (0.2, 0.6)$, $\beta = (0.6, 0.2)$ — both positive (see the results part).

The QAOA kernel, line by line (Section 7a)

```
@cudaq.kernel
def qaoa_state(n: int, edge_i: list[int],
               edge_j: list[int], weights: list[float],
               layers: int, params: list[float]):
    qubits = cudaq.qvector(n) # |0...0>
    h(qubits) # |+>^n
    for layer in range(layers):
        gamma = params[layer]
        beta = params[layers + layer]
        for e in range(len(edge_i)): # cost layer
            x.ctrl(qubits[edge_i[e]], qubits[edge_j[e]])
            rz(gamma * weights[e], qubits[edge_j[e]])
            x.ctrl(qubits[edge_i[e]], qubits[edge_j[e]])
        for q in range(n): # mixer layer
            rx(2.0 * beta, qubits[q])
```

- `@cudaq.kernel`: compiled to MLIR/QIR, then JIT-compiled ("compilation time" in the report).
- `h(qubits)` broadcasts H over the register.
- `params[layer]`, `params[layers+layer]`: the blocked layout $[\gamma, \beta]$.
- `x.ctrl(c, t)` is a CNOT; with `rz(gamma*w)`: $e^{-i\gamma \frac{w}{2} Z_i Z_j}$.
- `rx(2*beta)` $= e^{-i\beta X}$.
- `qaoa_sample` is the same body followed by `mz(qubits)`.

Objective, optimizer and sampling (Section 8)

```
def objective(params):
    result = cudaq.observe(kernels["state"], H_C, N_QUBITS, edge_i, edge_j, edge_w,
                           QAOA_LAYERS, list(params), **observe_kwargs) # exact, or shots
    energy = result.expectation() # <H_C>
    history_cut.append(total_weight / 2 - energy) # record <C>
    history_params.append(np.array(params, dtype=float))
    return energy # SciPy MINIMISES <H_C>

opt = minimize(objective, initial_params, method="COBYLA",
               options={"maxiter": MAX_EVALUATIONS, "rhobeg": 0.2})
best_params = history_params[int(np.argmax(history_cut))] # best evaluated angles
counts = cudaq.sample(kernels["sample"], N_QUBITS, edge_i, edge_j, edge_w,
                      QAOA_LAYERS, best_params.tolist(), shots_count=SHOTS)
```

`observe` returns the exact $\langle H_C \rangle$ on simulators (or a shot estimate if `OBJECTIVE_SHOTS` is set; always on hardware). COBYLA is derivative-free; `rhobeg` is its initial trust-region radius. Other choices: Nelder-Mead, Powell, L-BFGS-B (finite differences, $2p + 1$ runs per gradient).

Result extraction (Sections 8–9)

```
samples = sorted(((b, c, cut_value(b)) for b, c in counts.items()),
                 key=lambda t: (-t[1], -t[2])) # by frequency, then by cut
best_sample = max(samples, key=lambda t: (t[2], t[1])) # largest cut seen
optimal_hits = sum(c for b, c, v in samples if np.isclose(v, max_cut))
ratio = expected_cut / max_cut # r = <C> / C_max
```

- Each key of `counts` is a bitstring (qubit/node 0 first) with its number of shots.
- The cut of every sample is computed *classically* with `cut_value` — checking a candidate is cheap.
- Reported: best expected cut, r, best sampled split, P(optimal split), timing and memory.

Plots produced

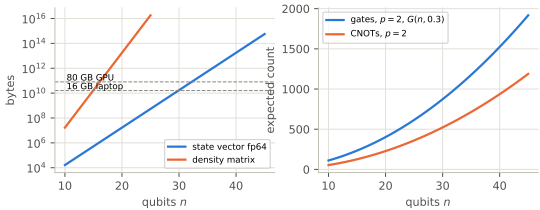
Objective convergence; best split drawn on the graph; most frequent bitstrings; distribution of cut values; memory timeline; final report figure.

Backends: CPU, GPU, multi-GPU, hardware

`BACKEND` + `SIMULATION_METHOD`	CUDA-Q target	memory	validated here?
`cpu` + `statevector`	`qpp-cpu`	16×2^n B	**yes** (recorded + re-run)
`cpu` + `density_matrix`	`density-matrix-cpu`	16×4^n B	no
`gpu` + `statevector`	`nvidia` (fp32/fp64)	8 / 16×2^n B	no (no GPU)
`gpu` + `mps` / `tensornet`	`tensornet-mps` / `tensornet`	depends on entanglement	no (no GPU)
`multi-gpu` + `statevector`	`nvidia`, option `mgpu` (MPI)	split over GPUs	no
`hardware`	`quantinuum`, `ionq`, ...	—	no

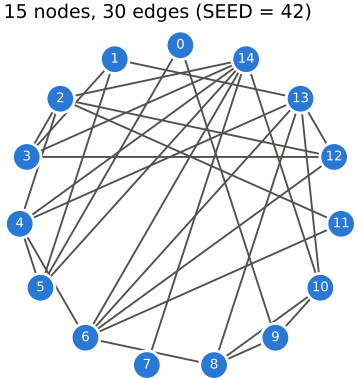
- `"auto"` picks a GPU if CUDA-Q sees one, else the CPU. Unsupported pairs stop with an explanation (e.g. `cpu` + `mps`: "needs an NVIDIA GPU").
- Multi-GPU pooling needs one MPI rank per GPU: `jupyter nbconvert -to script`, then `mpiexec -np 4`.
- Hardware targets return samples only: the objective is then always estimated from `SHOTS` shots. `HARDWARE_EMULATE = True` runs the provider's flow locally.

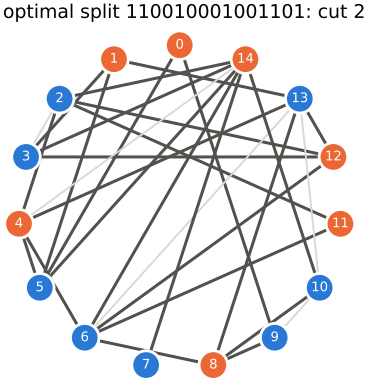
Memory, time estimates and exports



- Section 6 computes the state memory exactly and refuses runs above `MEMORY_SAFETY_FRACTION` (80 %).
- Time estimate: (evaluations) $\times$ (gates) $\times$ (measured time per gate), scaled by 2^n — rough, about $\pm 3\times$.
- Recorded: estimate $\sim$ 39.8 s, measured 40.8 s.
- Exports in `exports/qaoa/`: circuit drawing (`.txt`, `.png`), full circuit as QIR (`.ll`), report (`.json`, `.png`).

The default instance





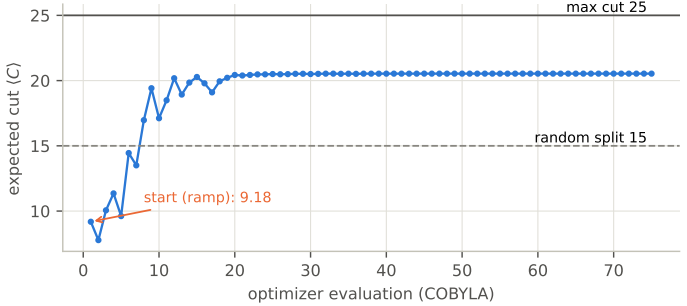
15 nodes, 30 unit edges, $W/2 = 15$. Exact optimum (brute force over 32 768 splits): $C_{\max} = 25$; 4 optimal bitstrings = 2 partitions (complementary pairs). Node 10 can join either side in one of them.

Recorded versus re-executed results

	previously recorded (notebook outputs)	**re-executed for this lecture**
machine	4 vCPU Xeon, 16.9 GB	laptop, WSL Ubuntu, 12 threads, 3.9 GB
CUDA-Q / target	0.16.0 / `qpp-cpu`	0.16.0 / `qpp-cpu`
COBYLA evaluations	80 (budget reached)	75 (trust region at lower bound)
best $\langle C \rangle$, r	20.5336, 0.8213	20.5336, 0.8213
best sampled cut	25 (110010001001101)	25 (110010001001101)
P(optimal split)	0.0600 (240 / 4000)	0.0600 (240 / 4000)
execution cell	40.8 s	183.9 s

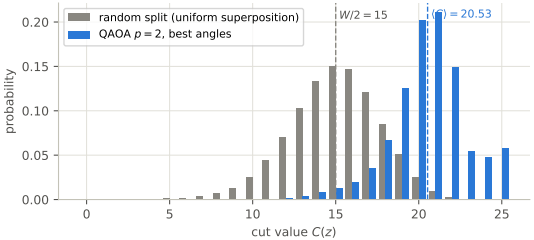
- Same seed, same versions: the optimisation results agree to all printed digits; the number of optimizer steps differs slightly (floating-point summation order with different thread counts).
- Timings are machine-specific. GPU, multi-GPU and hardware paths were **not** executed (no GPU available).
- The lecture's own script (`make_figures.py`) obtains $\langle C \rangle = 20.5336$, $r = 0.8213$ after 75 evaluations; CUDA-Q and NumPy states agree with fidelity 1.000000000000.

Convergence of the optimizer



- The **ramp start gives $\langle C \rangle = 9.18$ — below random guessing (15)**. Reason: $\gamma, \beta > 0$ in the notebook's convention heads towards *small* cuts (Part 6). COBYLA leaves this region within a few steps.
- Best angles found: $\boldsymbol{\gamma} = (0.367, 0.690)$, $\boldsymbol{\beta} = (1.117, -0.275)$. Because F_p has period $\pi/2$ in each β_l, $\beta_1 = 1.117 \equiv 1.117 - \pi/2 = -0.454$: the optimizer ended with $\gamma > 0$ and $\beta < 0$ — **opposite signs**, as the analysis predicts.

What the optimised circuit produces



- The distribution of cut values moves from the random bell curve (mean 15, $\sigma = 2.74$) to mean 20.53.
- $P(C = 25)$: 1.22×10^{-4} (random) $\rightarrow$ 0.0580 (QAOA, exact from the state vector; the 4000-shot estimate is 0.0600): about 475$\times$ more likely.
- The 4 most frequent strings are exactly the 4 optimal ones; complementary strings have nearly equal counts (68/67, 56/49) — the $z \leftrightarrow \bar{z}$ symmetry, up to shot noise.
- Mean cut over 4000 shots: 20.520 (exact $\langle C \rangle = 20.53$).

Interpreting the numbers correctly

Approximation ratio $r = 0.82$

An *average* over the output distribution. It does *not* mean 82 % of shots are optimal (6 % are).

Improvement over random

Random guessing already has $r = 15/25 = 0.6$. A stricter measure:

$$\eta = \frac{\langle C \rangle - W/2}{C_{\max} - W/2} = \frac{20.53 - 15}{10} = 0.553.$$

QAOA as a sampler

With $P_{\text{opt}} = 0.06$, 75 shots already give 99 % probability of seeing an optimum ($1 - 0.94^{75} \geq 0.99$). The best of 4000 shots was optimal.

No speed-up claim

This 15-node instance is brute-forced classically in 0.01 s. The value of the experiment is a reproducible benchmark of the method and of the simulator.

Performance analysis of the recorded run

phase (recorded)	time
circuit construction	0.019 s
compilation (MLIR + JIT)	0.045 s
quantum execution (80 `observe`)	40.01 s
final sampling (4000 shots)	0.60 s
optimizer + Python overhead	0.13 s
execution cell	40.84 s

- $40.01/80 \approx 0.50$ s per evaluation, ≈ 2.2 ms per gate — the calibration predicted 2.18 ms.
- Simulation dominates completely; the classical optimizer costs almost nothing.
- Peak host RAM 0.89 GB; the state itself is only 0.5 MB — the Python/CUDA-Q runtime dominates memory at this size.
- Model: $n = 25$, $|E| = 90$ would take $\approx 2.8 \times 10^3\times$ longer on the same CPU (≈ 31 h): use a GPU.

Limitations and challenges

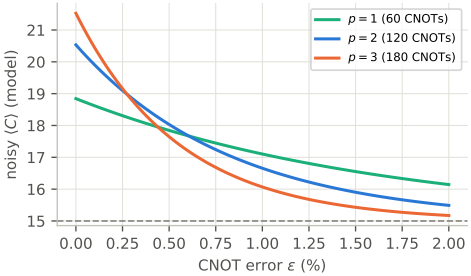
- **Optimisation landscape**: many local optima; results depend on the start point (see the ramp).
- **Gradients**: exact parameter-shift gradients need 2 runs *per gate* using the angle — $2p(|E| + n) = 180$ runs for the default, versus 5 for finite differences.
- **Shot noise**: on hardware each objective value has error $\sigma_C/\sqrt{S}$ (≈ 0.08 for $\sigma_C = 2.5$, $S = 1000$).
- **Locality**: fixed p sees only p-neighbourhoods; low-depth QAOA provably cannot reach the optimum on some graph families (Bravyi et al. 2020).
- **Guarantees**: $p = 1$ gives 0.6924 on 3-regular graphs; Goemans–Williamson gives 0.878 classically.
- **Classical simulation**: 16×2^n bytes — 32 qubits fit on one 80 GB GPU (fp64), 36 qubits need 16 such GPUs.
- **No demonstrated quantum advantage** for Max-Cut so far.

Noise and hardware considerations

- Each CNOT has an error rate ε (typically 10^{-3}–10^{-2} today).
- Illustrative global-depolarising model: with probability $F = (1-\varepsilon)^{N_{\mathrm{CNOT}}}$ the ideal output, otherwise a random bitstring:

$$\langle C \rangle_{\mathrm{noisy}} = F \langle C \rangle_{\mathrm{ideal}} + (1-F)\,\frac{W}{2}.$$

- Default (120 CNOTs, $\varepsilon = 0.5\,\%$): $F = 0.55$, $\langle C \rangle \approx 18.0$.
- **Connectivity**: on hardware with nearest-neighbour coupling, distant edges need SWAP gates (3 CNOTs each).
- Deeper circuits help in theory but cost fidelity: an optimal p exists for given hardware.



The model is an assumption for illustration, not a hardware measurement. `Noise_Simulation.ipynb` shows real CUDA-Q noise channels.

Advanced QAOA and current research

Better angles

- **INTERP** (Zhou et al. 2020): interpolate optimised depth-p angles to start depth $p+1$; e.g. $(0.4, 0.8) \to (0.4, 0.6, 0.8)$.
- **Parameter transfer / concentration**: optimal angles depend mainly on local graph structure, so angles from small graphs work on large ones.

Better ansatz

- **Warm-start QAOA**: initial state from a classical relaxation.
- **ADAPT-QAOA**: grow mixers from an operator pool by gradient.

Other problems

- **Quantum Alternating Operator Ansatz** (Hadfield et al. 2019): mixers such as the XY mixer that keep constrained problems (graph colouring, TSP) in the feasible subspace.
- **Recursive QAOA** (Bravyi et al. 2020): fix variables from strong correlations $\langle Z_i Z_j \rangle$ and recurse.

None of these variants is implemented in `QAOA.ipynb`; they are natural extensions of its code.

Try it yourself: `QAOA.ipynb`

1. Install: `pip install -r requirements.txt`
2. Open `QAOA.ipynb` and run all cells.
3. Change only Section 3:
   - `GRAPH = "ring"`, `N_QUBITS = 8`: compare r with $\frac{3}{4}$, $\frac{5}{6}$, ...
   - `INITIAL_PARAMETERS = [0.2, 0.6, -0.6, -0.2]`: start with opposite signs
   - `QAOA_LAYERS = 3`, `MAX_EVALUATIONS = 200`

Experiments

- `OBJECTIVE_SHOTS = 1000`: optimise with shot noise, as on hardware
- `OPTIMIZER = "L-BFGS-B"`: gradient-based (finite differences)
- `N_QUBITS = 24`, `BACKEND = "gpu"`: compare execution times
- `GRAPH = "custom"` with weights, including negative ones

The notebook checks every result against the brute-force optimum and reports time and memory.

Exercises

1. Write the cut of the square $0{-}1{-}2{-}3{-}0$ as a QUBO polynomial and as $\frac{W}{2} - H_C$.
2. How many bitstrings and how many partitions does a 20-node graph have?
3. Show that $\mathrm{CNOT}\,(I \otimes R_Z(\theta))\,\mathrm{CNOT}$ acts on $|11\rangle$ as $e^{-i\theta/2}$.
4. For one edge and $p = 1$, find all $(\gamma, \beta) \in [0, \pi] \times [-\frac{\pi}{4}, \frac{\pi}{4}]$ with $F_1 = 1$.
5. A ring C_{10} at $p = 1$ with optimal angles: what is the expected cut?
6. Gates and CNOTs of the notebook's circuit for $n = 12$, $|E| = 20$, $p = 3$?
7. Convert the textbook optimum $(\gamma, \beta) = (\frac{\pi}{4}, \frac{\pi}{8})$ for a ring into the notebook's convention.
8. With $P_{\text{opt}} = 0.02$, how many shots give a 95 % chance of at least one optimal sample?

Answers

1. $C = (z_0 + z_1 - 2z_0z_1) + \cdots = 2(z_0 + z_1 + z_2 + z_3) - 2(z_0z_1 + z_1z_2 + z_2z_3 + z_3z_0)$;
   $C = 2 - \frac{1}{2}(Z_0Z_1 + Z_1Z_2 + Z_2Z_3 + Z_3Z_0)$.
2. $2^{20} = 1\,048\,576$ bitstrings, $2^{19} = 524\,288$ partitions.
3. $|11\rangle \to |10\rangle$ (CNOT), R_Z gives $e^{-i\theta/2}$ (bit 0), CNOT back to $|11\rangle$: phase $e^{-i\theta/2}$, as for $e^{-i\theta Z\otimes Z/2}$ with $s_0s_1 = +1$.
4. Need $\sin 4\beta \sin\gamma = -1$: $\gamma = \frac{\pi}{2}$ and $\sin 4\beta = -1$, i.e. $\beta = -\frac{\pi}{8}$.
5. $10 \times \frac{3}{4} = 7.5$ (and $C_{\max} = 10$).
6. $12 + 3(60 + 12) = 228$ gates, $2 \cdot 3 \cdot 20 = 120$ CNOTs.
7. Flip the sign of γ: $(-\frac{\pi}{4}, \frac{\pi}{8})$, or equivalently $(\frac{\pi}{4}, -\frac{\pi}{8})$.
8. $1 - 0.98^S \geq 0.95 \Rightarrow S \geq \ln 0.05 / \ln 0.98 = 148.3$, so $S = 149$.

Summary

- Max-Cut: split a graph to cut as much edge weight as possible; NP-hard, 2^n candidates.
- Spins turn the cut into an Ising energy: $C = \frac{W}{2} - H_C$, $H_C = \sum \frac{w}{2} Z_i Z_j$ — maximising the cut is finding the ground state.
- QAOA alternates the **cost layer** $e^{-i\gamma H_C}$ (phases; CNOT–R_Z–CNOT per edge) and the **mixer** $e^{-i\beta \sum X}$ (interference; $R_X(2\beta)$), starting from $|+\rangle^{\otimes n}$.
- A classical optimizer tunes $2p$ angles; the final circuit is **sampled**.
- Exact at $p = 1$ for one edge; rings give $\frac{3}{4}, \frac{5}{6}, \frac{7}{8}$; signs and conventions matter.
- `QAOA.ipynb`: $r = 0.82$, optimum found, $P_{\text{opt}} = 0.06$ at $p = 2$ — reproduced here with CUDA-Q.

Further reading

- E. Farhi, J. Goldstone, S. Gutmann, *A Quantum Approximate Optimization Algorithm* (2014)
- Z. Wang et al., *QAOA for MaxCut: a fermionic view*, PRA 97 (2018)
- S. Hadfield et al., *From the QAOA to a Quantum Alternating Operator Ansatz* (2019)
- L. Zhou et al., *QAOA: performance, mechanism, and implementation*, PRX 10 (2020)
- NVIDIA CUDA-Q documentation
- This repository: `QAOA.ipynb`, `assessments/QAOA_*.pdf`